LAPORAN PRATIKUM
ALGORITMA DAN STRUKTUR DATA

BIG O NOTASION TIME & SPACE COMPLEXITY

NAMA	:	Anisah Nabila
NIM	:	2025573010125
KELAS	:	TI 1D
DOSEN	:	M. Reza zulman, S.ST.,M.Sc

PROGRAM STUDI TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE 2026

LEMBARAN PENGESAHAN

No Praktikum	:	01/TIK/TI 1D/algoritma &struktur data/2026
Nama Praktikum	:	praktikum algoritma dan struktur data
Pembuat Laporan	:	Anisah Nabila
NIM	:	2025573010125
Kelas	:	TI-1D
Jurusan	:	Teknologi Informasi dan komputer
Prodi	:	Teknik Informatika
Tanggal Praktikum	:	23 April 2026
Tanggal Penyerahan Laporan	:	28 April 2026

Lhokseumawe, 27 April 2026

penulis

ANISAH NABILA

NIM.2025573010125

KATA PENGANTAR

Puji syukur saya panjatkan ke hadirat Tuhan Yang Maha Esa, karena berkat rahmat dan karunia-Nya, laporan ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai bagian dari tugas dalam mata kuliah algoritma dan struktur data yang bertujuan untuk memberikan pemahaman dan praktek dalam pengelolaan dan pengorganisasian data secara sistematis. serta agar dapat mengingat langkah-langkah yang ada pada laporan ini jika sudah tidak mengingatnya di kemudian hari.

Dalam laporan ini, dibahas berbagai konsep dasar untuk memahami matari pada laporan, program,serta langkah-langkah, termasuk di dalamnya memakai beberapa aplikasi yang dibutuhkan. Selain itu, laporan ini juga mencakup aplikasi praktis dari teori yang telah dipelajari selama perkuliahan.

Saya menyadari bahwa laporan ini tidak akan terwujud tanpa bantuan dan dukungan dari berbagai pihak. Oleh karena itu, saya mengucapkan terima kasih yang sebesar-besarnya kepada dosen pembimbing mata kuliah algoritma dan struktur data, teman-teman, serta pihak-pihak yang telah memberikan bantuan baik secara langsung maupun tidak langsung dalam penyelesaian laporan ini.

Semoga laporan ini dapat memberikan manfaat dan menambah wawasan bagi semua pihak yang membacanya. Kritik dan saran yang konstruktif sangat diharapkan untuk penyempurnaan laporan ini di masa mendatang.

DAFTAR ISI

LEMBARAN PENGESAHAN 2
KATA PENGANTAR 3
DAFTAR ISI 4
BAB 1 PENDAHULUAN 5
1.1 LATAR BELAKANG 5
1.2 RUMUSAN MASALAH 5
1.3 TUJUAN PENULISAN 6
1.4 MANFAAT PENULISAN 6
BAB ll LANDASAN TEORI 7
2.1 BIG NOTATION TIME & SPACE COMLEXITY 7
2.2 FUNGSI NOTASI BIG O 7
2.3 JENIS-JENIS KOMPLEKSITAS NOTASI BIG O 7
2.4 KONSEP PENTING DALAM BIG O 8
2.5 PERBEDAAN TIME COMPLEXCITY DAN SPACE COMPLEXCITY 9
BAB III PEMBAHASAN 10
3.1 LANGKAH KERJA SELAMA PROSES PRAKTIKUM 10
3.2 LATIHAN 15
3.3 TUGAS 18
BAB IV ANALISA HASIL 22
BAB V PENUTUP 24
5.1 KESIMPULAN 24
5.2 SARAN 24
DAFTAR PUSTAKA 25

BAB 1 PENDAHULUAN

1.1 LATAR BELAKANG

Dalam dunia pemrograman dan ilmu komputer, efisiensi suatu algoritma menjadi hal yang sangat penting, terutama ketika menangani data dalam jumlah besar. Seiring berkembangnya teknologi, kebutuhan akan sistem yang cepat dan hemat sumber daya semakin meningkat. Oleh karena itu, diperlukan suatu cara untuk mengukur dan menganalisis kinerja algoritma agar dapat memilih solusi yang paling optimal.

Salah satu konsep yang digunakan untuk menganalisis kinerja algoritma adalah *Big O Notation*. Big O Notation memberikan gambaran tentang seberapa cepat atau lambat suatu algoritma berjalan seiring dengan bertambahnya ukuran input. Dengan menggunakan notasi ini, seorang programmer dapat membandingkan beberapa algoritma dan menentukan mana yang lebih efisien tanpa harus menjalankan program tersebut secara langsung.

Selain itu, terdapat dua aspek penting dalam analisis algoritma, yaitu *Time Complexity* dan *Space Complexity*. Time Complexity digunakan untuk mengukur waktu eksekusi yang dibutuhkan oleh suatu algoritma, sedangkan Space Complexity mengukur jumlah memori yang digunakan selama proses eksekusi. Kedua aspek ini sangat berpengaruh dalam pengembangan perangkat lunak, terutama pada sistem yang memiliki keterbatasan waktu dan sumber daya.

Pemahaman mengenai Big O Notation, Time Complexity, dan Space Complexity sangat penting bagi mahasiswa maupun praktisi di bidang teknologi informasi. Dengan memahami konsep ini, seseorang dapat merancang algoritma yang lebih efisien, mengoptimalkan performa program, serta menghindari penggunaan sumber daya yang berlebihan. Oleh karena itu, materi ini menjadi dasar yang penting dalam pembelajaran struktur data dan algoritma.

1.2 RUMUSAN MASALAH

Adapun rumusan masalah Adalah sebagai berikut :

1. Apa yang dimaksud dengan Big O Notation dalam analisis algoritma?
2. Bagaimana cara kerja dan fungsi Time Complexity dalam mengukur efisiensi algoritma?

3. Apa yang dimaksud dengan Space Complexity dan bagaimana pengaruhnya terhadap penggunaan memori?
4. Mengapa analisis kompleksitas (waktu dan ruang) penting dalam pengembangan program?
5. Bagaimana cara membandingkan efisiensi beberapa algoritma menggunakan Big O Notation?

1.3 TUJUAN PENULISAN

Adapun tujuan penulisan laporan ini adalah sebagai berikut :

1. Untuk memahami konsep dasar Big O Notation dalam analisis algoritma.
2. Untuk mengetahui cara mengukur waktu eksekusi algoritma melalui Time Complexity.
3. Untuk memahami penggunaan memori algoritma melalui Space Complexity.
4. Untuk meningkatkan kemampuan dalam memilih algoritma yang efisien dan optimal.
5. Untuk memberikan dasar pengetahuan dalam pengembangan program yang lebih efektif dan efisien.

1.4 MANFAAT PENULISAN

Adapun manfaat penulisan laporan ini adalah sebagai berikut :

1. Memberikan pemahaman tentang pentingnya efisiensi algoritma dalam pemrograman
2. Membantu dalam menentukan algoritma terbaik berdasarkan kebutuhan sistem
3. Mengurangi penggunaan waktu dan memori yang tidak efisien dalam program.
4. Menjadi dasar dalam mempelajari struktur data dan algoritma yang lebih kompleks.
5. Meningkatkan kemampuan analisis dan logika dalam menyelesaikan permasalahan komputasi.

BAB ll LANDASAN TEORI

2.1 BIG NOTATION TIME & SPACE COMLEXITY

Notasi Big O adalah cara untuk mengukur seberapa efisien suatu algoritma berdasarkan pertumbuhan jumlah data (input). Notasi ini menunjukkan batas atas (worst-case) dari waktu eksekusi atau penggunaan memori suatu algoritma. Dengan kata lain, Big O digunakan untuk menjawab :"Seberapa cepat algoritma berjalan ketika data semakin besar?"

Notasi Big O adalah notasi matematika yang menggambarkan perilaku limit suatu fungsi ketika argumennya cenderung menuju nilai tertentu atau tak hingga. Notasi ini merupakan bagian dari keluarga notasi yang diciptakan oleh Paul Bachmann, Edmund Landau, dan lainnya, yang secara kolektif disebut notasi Bachmann–Landau atau notasi asimtotik.

Sederhananya, notasi Big O menggambarkan **kompleksitas** kode Anda menggunakan istilah aljabar. Untuk memahami apa itu notasi Big O, kita dapat melihat contoh tipikal, ***O(n²)*** , yang biasanya diucapkan ***"Big O kuadrat"*** . Huruf ***"n"*** di sini mewakili **ukuran input** , dan fungsi ***"g(n) = n²"*** di dalam ***"O()"*** memberi kita gambaran tentang seberapa kompleks algoritma tersebut terhadap ukuran input.

Algoritma tipikal yang memiliki kompleksitas O(n²) adalah algoritma **selection sort** . Selection sort adalah algoritma pengurutan yang berulang melalui daftar untuk memastikan setiap elemen pada indeks ***i*** adalah elemen terkecil/terbesar ***ke-i*** dalam daftar tersebut.

2.2 FUNGSI NOTASI BIG O

- Membandingkan efisiensi beberapa algoritma
- Mengetahui performa algoritma pada data besar
- Membantu memilih algoritma terbaik
- Mengoptimalkan program agar lebih cepat dan hemat memori

2.3 JENIS-JENIS KOMPLEKSITAS NOTASI BIG O

Berikut adalah beberapa jenis kompleksitas yang paling umum:

- **O(1) – Constant Time** → waktu eksekusi tidak tergantung jumlah data.

Contoh : int x = arr[0]; → selalu cepat, tidak peduli ukuran array.

- **O(n) – Linear Time →** Waktu eksekusi bertambah sebanding dengan jumlah data.

Contoh: for(i = 0; I < n; i++) {

Printf(" %d", arr[i]); → semakin banyak data semakin lama.

}

- **O(log n) – Logarithmic Time →** Waktu eksekusi bertambah lebih lambat dari linear (biasanya membagi data).

Contoh : Binary Search → Sangat efisien untuk data besar.

- **O(n^2) – Quadratic Time →** Waktu eksekusi bertambah sangat cepat karena ada perulangan bersarang.
- **O(2^n) – Exponential Time →** Waktu eksekusi meningkat sangat cepat (eksponensial).

Contoh : Rekursi Fibonacci tanpa optimasi → Sangat lambat untuk input besar.

Urutan Kompleksitas (Terbaik ke Terburuk)

- O(1) → O(log n) → O(n) → O(n log n) → O(n^2) → O(2^n).

Notasi Big O adalah alat penting untuk menganalisis efisiensi algoritma. Dengan memahami Big O, kita dapat membuat program yang lebih cepat, efisien, dan optimal, terutama saat menangani data dalam jumlah besar.

2.4 KONSEP PENTING DALAM BIG O

- **Worst Case, Average Case, Best Case**

Dalam analisis algoritma, ada 3 kemungkinan kondisi :

1. Worst case (kasus terburuk) → kondisi paling lama.

Contoh : mencari data yang tidak ada di array.

2. Avarage Case (Rata-rata) → kondisi umum.
3. Best Case (Kasus Terbaik) → kondisi paling cepat.

Contoh : data ditemukan di indeks pertama.

Big O biasanya focus ke Worst Case, karena ini yang paling aman untuk mengukur performa.

2.5 PERBEDAAN TIME COMPLEXCITY DAN SPACE COMPLEXCITY

- Time complexity → mengukur waktu eksekusi

Contoh : O(n), O(n^2).

- Space complexity → mengukur penggunaan memori

Contoh : int arr[100];

Cara memahami big O notasi dengan cepat ialah dengan mengingat cara berikut :

3 Satu loop → O(n)

4 Dua loop bersarang → $O(n^2)$

5 Pembagian berulang → O(log n)

6 Rekursi bercabang → biasanya eksponensial

7 Abaikan angka kecil dan konstanta

Big O Notation tidak menghitung waktu sebenarnya, tetapi mengukur pertumbuhan performa algoritma. Semakin kecil kompleksitasnya, semakin efisien algoritma tersebut untuk data besar.

BAB III PEMBAHASAN

3.1 LANGKAH KERJA SELAMA PROSES PRAKTIKUM

1. Buka aplikasi GitBash, ketik cd dan copy repository anda di file kemudian paste di samping tulisan cd pada GitBash.

```
MINGW64:/d/New folder/praktikum_strukturdata/2025573010125_strukturdatadanalgoritma/praktikum-5

ACER@DESKTOP-2O4TJ2F MINGW64 ~
$ cd '/d/New folder/praktikum_strukturdata/2025573010125_strukturdatadanalgoritm
a'
```

2. Buat folder praktikum-5 dan masuk kedalamnya dengan cara ketik mkdir praktikum-5 kemudian enter, lalu ketik cd praktikum-5 seperti gambar berikut.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ mkdir praktikum-5

ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ cd praktikum-5
```

3. Inisialisasi npm project dengan cara ketik npm init -y pada GitBash.

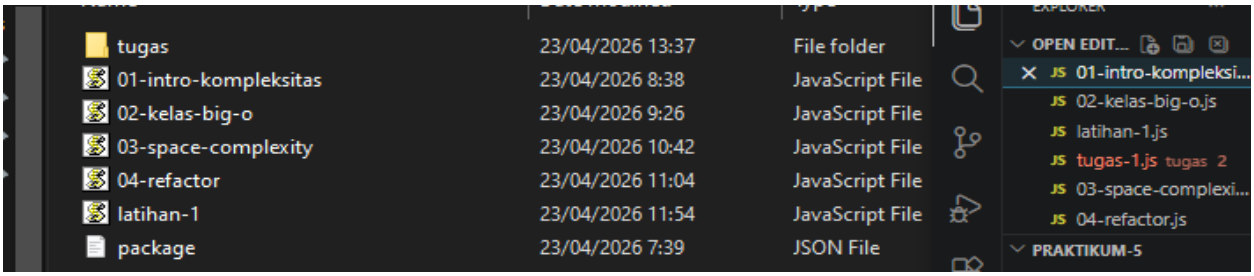
```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ npm init -y
Wrote to D:\New folder\praktikum_strukturdata\2025573010125_strukturdatadanalgor
itma\praktikum-5\package.json:

{
  "name": "praktikum-5",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}
```

4. Pastikan kamu berada di dalam folder praktikum-5, kemudia buat file baru dengan cara ketik touch 01-intro-kompleksitas.js.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch 01-intro-kompleksitas.js
```

Dengan membuat file baru di node.js maka akan otomatis ter-save dan muncul pada file manager maupun pada vs.code file baru tersebut.



5. setelah membuat latihan dan tugas simpan file, kemudian jalankan file tersebut dari terminal dengan mengetik node 01-intro-kompleksitas.js.dan akan langsung keluar tampilannya.
6. Buat file baru didalam folder praktikum-5 dengan cara ketik touch 02-kelas-big-o.js di GitBash.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch 02-kelas-big-o.js
```

7. Simpan file dan jalankan dari terminal.
8. Buat file baru dengan cara ketik touch 03-space-complexity.js dan jangan lupa pastikan anda masih berada dalam folder praktikum-5 di GitBash.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch 03-space-complexity.js
```

9. Buka file 03-space-complexity.js di vs code lalu kerjakan latihan yang ada pada modul, latihannya akan saya tampilkan di bawah setelah selesai langkah-langkah.
10. Simpan file kemudian jalankan dari terminal ketikkan node 03-space-complexity.js maka akan keluar tampilannya.
11. Buat file baru didalam folder praktikum-5 dengan cara ketik touch 04-refactor.js di GitBash.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch 04-refactor.js
```

12. Simpan file lalu jalankan dari terminal.
13. Jangan lupa kerjakan latihan dan tugas yang ada didalam modul.
14. Buat file baru dengan cara ketik touch latihan-1.js

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch latihan-1.js
```

15. Buka file latihan-1.js lalu kerjakan latihan yang ada di dalam modul.
16. Simpan file lalu jalankan dari terminal.
17. Buat folder tugas di dalam praktikum-5, kemudian kerjakan kedua tugas berikut, akan saya tampilkan tugasnya di bawah setelah langkah-langkah dibawah ini.
18. Buat folder tugas dengan dua file tugas di dalamnya dengan cara ketik mkdir tugas.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ mkdir tugas
```

19. Buat file tugas didalam folder tugas dengan cara ketik seperti gambar dibawah pada GitBash.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma/praktikum-5 (main)
$ touch tugas/tugas-1.js tugas/tugas-2.js
```

20. Buat tugas 1 dan 2 pada vs.code kemudian jalankan dari terminal.
21. Setelah latihan dan tugas selesai dikerjakan langkah selanjutnya ialah push ke repository GitHub anda.
22. Pastikan berada di root folder repository(bukan di dalam praktikum-5), lalu cek status file.
23. Buka GitBash kemudian ketik cd dan copy repository anda lalu enter.

```
ACER@DESKTOP-2O4TJ2F MINGW64 ~
$ cd '/d/New folder/praktikum_strukturdata/2025573010125_strukturdatadanalgoritm
a'
```

24. Ketik git status.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   praktikum-3/tugas/tugas-1.js
        modified:   praktikum-4/01-object-literal.js
        modified:   praktikum-4/tugas/tugas-1.js

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        praktikum-5/

no changes added to commit (use "git add" and/or "git commit -a")
```

25. Tambahkan seluruh folder ke praktikum-5 ke staging area.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ git add praktikum-5/
warning: in the working copy of 'praktikum-5/package.json', LF will be replaced
by CRLF the next time Git touches it
```

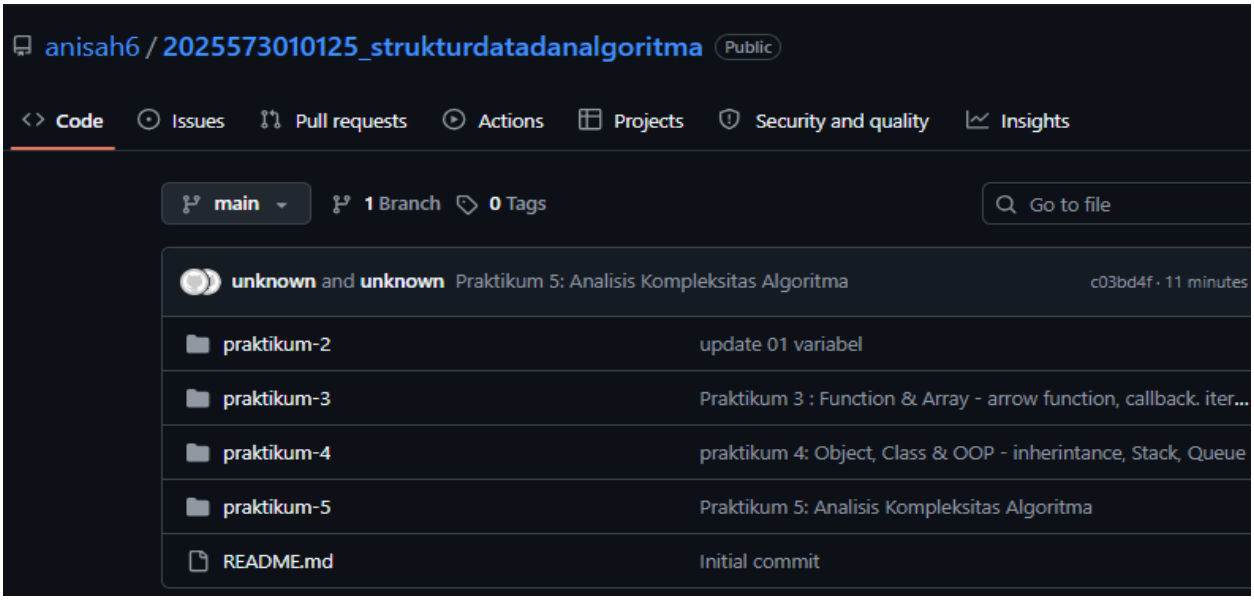
26. Commit dan push dengan cara ketik seperti gambar dibawah.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ git commit -m "Praktikum 5: Analisis Kompleksitas Algoritma"
[main c03bd4f] Praktikum 5: Analisis Kompleksitas Algoritma
 8 files changed, 454 insertions(+)
 create mode 100644 praktikum-5/01-intro-kompleksitas.js
 create mode 100644 praktikum-5/02-kelas-big-o.js
 create mode 100644 praktikum-5/03-space-complexity.js
 create mode 100644 praktikum-5/04-refactor.js
 create mode 100644 praktikum-5/latihan-1.js
 create mode 100644 praktikum-5/package.json
 create mode 100644 praktikum-5/tugas/tugas-1.js
 create mode 100644 praktikum-5/tugas/tugas-2.js
```

27. Push ke repository Github dan ketik git push origin main.

```
ACER@DESKTOP-2O4TJ2F MINGW64 /d/New folder/praktikum_strukturdata/2025573010125_
strukturdatadanalgoritma (main)
$ git push origin main
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 8 threads
Compressing objects: 100% (12/12), done.
Writing objects: 100% (12/12), 4.87 KiB | 1.22 MiB/s, done.
Total 12 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To https://github.com/anisah6/2025573010125_strukturdatadanalgoritma.git
   cfa8866..c03bd4f  main -> main
```

28. Lihat tugas yang sudah anda push ke GitHub.



Filter files...

- praktikum-5
  - 01-intro-kompleksitas.js
  - 02-kelas-big-o.js
  - 03-space-complexity.js
  - 04-refactor.js
  - latihan-1.js
  - package.json
- tugas
  - tugas-1.js
  - tugas-2.js

Search within code

praktikum-5/01-intro-kompleksitas.js

```
@@ -0,0 +1,36 @@
+ function ukurWaktu(label, fn) {
+     const mulai = Date.now();
+     fn();
+     const selesai = Date.now();
+     console.log(`${label}: ${selesai - mulai} ms`);
+ }
+
+ const N = 100_000; // ukuran data: 100 ribu
+
+ function jumlahkanLinear(n) {
+     let total = 0;
+     for (let i = 1; i <= n; i++) total += i;
+ }
+
+ function jumlahkanRumus(n) {
```

Code | Issues | Pull requests | Actions | Projects | Security and quality | Insights

Commit c03bd4f

unknown authored and **unknown** committed 14 minutes ago

Praktikum 5: Analisis Kompleksitas Algoritma

main

8 files changed

Filter files...

- praktikum-5
  - 01-intro-kompleksitas.js
  - 02-kelas-big-o.js
  - 03-space-complexity.js
  - 04-refactor.js
  - latihan-1.js
  - package.json
- tugas

Search within code

praktikum-5/01-intro-kompleksitas.js

```
@@ -0,0 +1,36 @@
+ function ukurWaktu(label, fn) {
+     const mulai = Date.now();
+     fn();
+     const selesai = Date.now();
+     console.log(`${label}: ${selesai - mulai} ms`);
+ }
+
```

29. setelah semua selesai dan silahkan lanjut membuat laporan praktikum struktur data dan algoritma anda.

3.2 LATIHAN

LATIHAN 1 → Identifikasi Kompleksit

```
JS latihan-1.js > ...
function fnA(n) {
    return n * 2;
}

function fnB(n) {
    for (let i = 0; i < n; i++){
        for (let j = 0; j < n; j++) {
            console.log(i, j);
        }
    }
}

function fnC(n) {
    for (let i = 1; i < n; i *= 2) {
        console.log(i);
    }
}

function fnD(arr) {
    arr.forEach(x => {
        arr.forEach(y => {
        arr.forEach(z => {
            console.log(x, y, z);
        });
    });
});
}

function hitungKompleksitas(n, fn, tipe = "n") {
    const start = Date.now();

    if (tipe === 'array') {
        const arr = Array.from({ length:n}, (_, i) => i);
        fn(arr);
    } else {
        fn(n);
    }

    const end = Date.now();
    console.log(`Waktu eksekusi: 4{end - start} ms\n`);
}
```

```
console.log("Fungsi A: ");
hitungKompleksitas(1000, fnA);

console.log("Fungsi B: ");
hitungKompleksitas(1000, fnB);

console.log("Fungsi C: ");
hitungKompleksitas(1000, fnC);

console.log("Fungsi D: ");
hitungKompleksitas(1000, fnD, "array");
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
4 97
4 98
4 99
4 100
4 101
4 102
4 103
4 104
4 105
4 106
4 107
```

LATIHAN 3 → Class BankAccount

```
console.log('Latihan : 3 Class BankAccount');

function cariPasanganLambat(arr, target) {
    for(let i = 0; i < arr.length; i++) {
        for(let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[i] === target) {
                return [arr[i], arr[i]];
            }
        }
    }
    return null;
}

function cariPasanganCepat(arr, target) {
    let seen = new Set();

    for (let num of arr) {
        let pasangan = target - num;

        if (seen.has(pasangan)) {
            return [pasangan, num];
        }
        seen.add(num);
    }
    return null;
}
```

```javascript

let data = [2, 7, 11, 15];
let target = 9;

console.log("Hasil Lambat :",
    cariPasanganLambat(data, target));
    console.log("Hasil Cepat :", cariPasanganCepat(data,target));

    let arrayBesar = [];

    for (let i = 0; i < 5000; i++) {
        arrayBesar.push(Math.floor(Math.random() * 100000));
    }

    let targetTidakAda = -1;

    console.time("Waktu Lambat");
    cariPasanganLambat(arrayBesar, targetTidakAda);
    console.timeEnd("Waktu Lambat");

    console.time("Waktu Cepat");
    cariPasanganCepat(arrayBesar, targetTidakAda);
    console.timeEnd("Waktu Cepat");

    console.log("/nKesimpulan:");
    console.log("Fungsi cepat lebih baik untuk data besar karena O(n).");
    console.log("Trade-off: membutuhkan memori tambahan untuk Set.");
    console.log("Fungsi lambat tidak butuh memori tambahan, tapi sangat lambat untuk data besar.");
```

```
Latihan : 3 Class BankAccount
Hasil Lambat : null
> Hasil Cepat : (2) [2, 7]
Waktu Lambat: 35.8720703125 ms
Waktu Lambat: 35.943ms
Waktu Cepat: 1.1650390625 ms
Waktu Cepat: 1.283ms
/nKesimpulan:
Fungsi cepat lebih baik untuk data besar karena O(n).
Trade-off: membutuhkan memori tambahan untuk Set.
Fungsi lambat tidak butuh memori tambahan, tapi sangat lambat untuk data besar.
```

3.3 TUGAS

TUGAS 1

tugas > JS tugas-1.js > ...

```javascript
function intersectionN2(arr1, arr2) {
    let result = [];
    for (let i = 0; i < arr1.length; i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j] && !result.includes(arr1[i])) {
                result.push(arr1[i]);
            }
        }
    }
    return result;
}

// O(n) pakai Set
function intersectionSet(arr1, arr2) {
    let set2 = new Set(arr2);
    return [...new Set(arr1.filter(x => set2.has(x)))];
}
// Time: O(n), Space: O(n)

function groupAnagrams(arr) {
    let map = {};

    for (let word of arr) {
        let key = word.split('').sort().join('');
        if (!map[key]) {
            map[key] = [];
        }
        map[key].push(word);
    }

    return Object.values(map);
}

function pythagorasN3(arr) {
    let n = arr.length;

    for (let i = 0; i < n; i++) {
        for (let j = i + 1; j < n; j++) {
            for (let k = 0; k < n; k++) {
                if (arr[i] + arr[j] === arr[k] * arr[k]) {
                    return true;
                }
```

```javascript
            }
        }
    }
    return false;
}

function pythagorasOptimized(arr) {
    arr.sort((a, b) => a - b);
```

```javascript

    let n = arr.length;

    for (let k = n - 1; k >= 0; k--) {
        let left = 0;
        let right = k - 1;

        while (left < right) {
            let sum = arr[left] + arr[right];
            let target = arr[k] * arr[k];

            if (sum === target) return true;
            else if (sum < target) left++;
            else right--;
        }
    }
    return false;
}

console.log("=== Fungsi A ===");
console.log(intersectionN2([1,2,3], [2,3,4]));
console.log(intersectionSet([1,2,3], [2,3,4]));

console.log("\n=== Fungsi B ===");
console.log(groupAnagrams(['eat','tea','tan','ate','nat','bat']));

console.log("\n=== Fungsi C ===");
console.log(pythagorasN3([1,2,3,4,5]));
console.log(pythagorasOptimized([1,2,3,4,5]));
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

C:\Program Files\nodejs\node.exe .\tugas\tugas-1.js
=== Fungsi A ===
> (2) [2, 3]
> (2) [2, 3]

=== Fungsi B ===
> (3) [Array(3), Array(2), Array(1)]

=== Fungsi C ===
true
false
```

TUGAS 2

```javascript
tugas > JS tugas-2.js > ...
function fn_O1(n) {
    return n + 1;
}

// O(n)
function fn_On(n) {
    let sum = 0;
    for (let i = 0; i < n; i++) {
        sum += i;
    }
    return sum;
}
```

```
// O(n log n)
function fn_OnLogn(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) {
            count++;
        }
    }
    return count;
}

// O(n^2)
function fn_On2(n) {
    let count = 0;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            count++;
        }
    }
    return count;
}
```

```
function benchmarkSemua(ukuranData) {
    ukuranData.forEach(n => {
        console.log(`\nUkuran Data: ${n}`);

        console.time("O(1)");
        fn_O1(n);
        console.timeEnd("O(1)");

        console.time("O(n)");
        fn_On(n);
        console.timeEnd("O(n)");

        console.time("O(n log n)");
        fn_OnLogn(n);
        console.timeEnd("O(n log n)");

        console.time("O(n^2)");
        fn_On2(n);
        console.timeEnd("O(n^2)");
    });
}

benchmarkSemua([100, 500, 1000, 5000, 10000]);

/*
KESIMPULAN:
- O(1) waktu selalu konstan
- O(n) bertambah linear
- O(n log n) lebih cepat dari O(n^2)
- O(n^2) sangat lambat saat data besar
=> Hasil sesuai teori Big O
*/
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    T
C:\Program Files\nodejs\node.exe

Ukuran Data: 100
O(1): 0.156005859375 ms
O(1): 0.484ms
O(n): 0.153076171875 ms
O(n): 0.707ms
O(n log n): 0.27099609375 ms
O(n log n): 0.456ms
O(n^2): 1.841064453125 ms
O(n^2): 2.027ms

Ukuran Data: 500
O(1): 0.156005859375 ms
O(1): 0.497ms
O(n): 0.072998046875 ms
O(n): 0.292ms
O(n log n): 0.55615234375 ms
O(n log n): 0.727ms
O(n^2): 1.68408203125 ms
O(n^2): 1.892ms
```

```
Ukuran Data: 1000
O(1): 0.049072265625 ms
O(1): 0.213ms
O(n): 0.120849609375 ms
O(n): 0.271ms
O(n log n): 0.236083984375 ms
O(n log n): 0.356ms
O(n^2): 2.192138671875 ms
O(n^2): 2.963ms

Ukuran Data: 5000
O(1): 0.027099609375 ms
O(1): 0.117ms
O(n): 0.031982421875 ms
O(n): 0.125ms
O(n log n): 0.343017578125 ms
O(n log n): 0.443ms
O(n^2): 53.30712890625 ms
O(n^2): 53.405ms
```

```
Ukuran Data: 10000
O(1): 0.009765625 ms
O(1): 0.047ms
O(n): 0.02294921875 ms
O(n): 0.071ms
O(n log n): 0.324951171875 ms
O(n log n): 0.388ms
O(n^2): 218.43994140625 ms
O(n^2): 218.53ms
```

BAB IV ANALISA HASIL

Berdasarkan pembahasan yang telah dilakukan mengenai Big O Notation, Time Complexity, dan Space Complexity, dapat dianalisis bahwa peran analisis algoritma sangat krusial dalam menentukan kualitas dan performa suatu program. Big O Notation memberikan pendekatan yang sistematis untuk mengevaluasi bagaimana suatu algoritma akan bekerja ketika dihadapkan pada pertumbuhan ukuran input yang semakin besar. Dengan demikian, analisis ini tidak hanya berfokus pada hasil akhir program, tetapi juga pada efisiensi proses yang terjadi di dalamnya.

Dari hasil pengamatan terhadap berbagai bentuk kompleksitas, terlihat bahwa algoritma dengan kompleksitas rendah seperti $O(1)$ dan $O(\log n)$ memiliki performa yang sangat stabil dan tidak terpengaruh secara signifikan oleh pertambahan jumlah data. Hal ini menjadikan algoritma dengan kompleksitas tersebut sangat ideal untuk digunakan dalam sistem yang membutuhkan kecepatan tinggi dan respons yang konsisten. Sebaliknya, algoritma dengan kompleksitas yang lebih tinggi seperti $O(n^2)$ dan $O(2^n)$ menunjukkan peningkatan waktu eksekusi yang sangat drastis seiring dengan bertambahnya ukuran input, sehingga menjadi kurang efisien untuk digunakan pada skala data yang besar.

Selain itu, hasil analisis menunjukkan bahwa struktur logika dalam penulisan algoritma sangat mempengaruhi nilai kompleksitasnya. Penggunaan perulangan tunggal umumnya menghasilkan kompleksitas linear, sedangkan perulangan bersarang secara langsung meningkatkan kompleksitas menjadi kuadratik atau bahkan lebih tinggi. Hal ini menegaskan bahwa perancangan algoritma yang baik harus mempertimbangkan jumlah operasi yang dilakukan secara keseluruhan, bukan hanya sekadar menghasilkan output yang benar.

Dalam konteks rekursi, analisis menunjukkan bahwa tanpa pengelolaan yang baik, seperti penggunaan teknik optimasi (misalnya memoization), rekursi dapat menyebabkan kompleksitas yang sangat tinggi, terutama pada kasus eksponensial. Hal ini sering terjadi pada algoritma yang melakukan pemanggilan fungsi berulang dengan parameter yang sama, sehingga menyebabkan redundansi perhitungan. Oleh karena itu, pemahaman terhadap pola rekursi dan cara mengoptimalkannya menjadi hal yang penting dalam meningkatkan efisiensi algoritma.

Selanjutnya, hubungan antara Time Complexity dan Space Complexity juga menjadi bagian penting dalam analisis. Dalam beberapa kasus, peningkatan penggunaan memori dapat membantu mengurangi waktu eksekusi, seperti pada penggunaan struktur data tambahan untuk menyimpan hasil sementara. Namun, pada sistem dengan keterbatasan memori, hal ini justru dapat menjadi kendala. Oleh

karena itu, diperlukan keseimbangan yang tepat antara penggunaan waktu dan ruang agar algoritma dapat berjalan secara optimal sesuai dengan kebutuhan dan batasan sistem.

Analisis juga menunjukkan bahwa Big O Notation tidak mengukur waktu eksekusi secara absolut, melainkan pertumbuhan relatif terhadap ukuran input. Hal ini membuat Big O menjadi alat yang sangat efektif untuk membandingkan algoritma secara umum, tanpa bergantung pada faktor eksternal seperti spesifikasi perangkat keras atau bahasa pemrograman yang digunakan. Dengan kata lain, Big O memberikan gambaran yang lebih universal tentang efisiensi algoritma.

Lebih lanjut, penerapan konsep Big O dalam pengembangan perangkat lunak dapat membantu dalam proses pengambilan keputusan, terutama dalam memilih algoritma yang tepat untuk suatu permasalahan. Dalam praktiknya, sering kali terdapat lebih dari satu solusi untuk menyelesaikan suatu masalah, sehingga analisis kompleksitas menjadi faktor penentu dalam memilih solusi yang paling efisien dan skalabel.

Secara keseluruhan, hasil analisis ini menegaskan bahwa pemahaman terhadap Big O Notation, Time Complexity, dan Space Complexity tidak hanya bersifat teoritis, tetapi juga memiliki implikasi praktis yang sangat besar dalam dunia pemrograman. Kemampuan untuk menganalisis dan mengoptimalkan algoritma menjadi keterampilan penting yang harus dimiliki oleh setiap programmer, terutama dalam menghadapi tantangan pengolahan data yang semakin kompleks dan besar di era digital saat ini.

BAB V PENUTUP

5.1 KESIMPULAN

Dari pembahasan yang telah dilakukan, dapat disimpulkan bahwa Big O Notation merupakan metode yang sangat penting dalam menganalisis kinerja algoritma, khususnya dalam mengukur pertumbuhan waktu eksekusi dan penggunaan memori terhadap ukuran input. Dengan memahami konsep Time Complexity dan Space Complexity, seorang programmer dapat menentukan algoritma yang paling efisien untuk digunakan dalam suatu permasalahan.

Pemahaman terhadap Big O juga membantu dalam menghindari penggunaan algoritma yang tidak efisien, terutama ketika berhadapan dengan data dalam jumlah besar. Semakin kecil nilai kompleksitas suatu algoritma, maka semakin baik performanya dalam jangka panjang. Oleh karena itu, penguasaan konsep ini menjadi dasar penting dalam pengembangan perangkat lunak yang optimal dan berkualitas.

5.2 SARAN

Agar pemahaman mengenai Big O Notation, Time Complexity, dan Space Complexity semakin baik, disarankan untuk lebih sering melakukan latihan analisis algoritma dari berbagai contoh kasus. Selain itu, mahasiswa atau pembelajar juga perlu membiasakan diri dalam menulis kode yang tidak hanya benar secara fungsi, tetapi juga efisien dari segi waktu dan memori.

Disarankan pula untuk mempelajari berbagai algoritma populer seperti searching dan sorting, serta membandingkan kompleksitasnya untuk mengetahui kelebihan dan kekurangannya. Penggunaan tools atau visualisasi algoritma juga dapat membantu dalam memahami bagaimana suatu algoritma bekerja.

Terakhir, penting untuk selalu mempertimbangkan kebutuhan sistem sebelum memilih algoritma, sehingga dapat mencapai keseimbangan antara performa dan penggunaan sumber daya yang tersedia.

DAFTAR PUSTAKA

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, & Clifford Stein. *Introduction to Algorithms* (3rd ed.). MIT Press, 2009.
2. Michael T. Goodrich & Roberto Tamassia. *Algorithm Design and Applications*. Wiley, 2014.
3. Robert Sedgewick & Kevin Wayne. *Algorithms* (4th ed.). Addison-Wesley, 2011.
4. GeeksforGeeks. "Analysis of Algorithms | Big O Notation." Diakses dari: https://www.geeksforgeeks.org
5. Khan Academy. "Asymptotic Notation (Big O)." Diakses dari: https://www.khanacademy.org
6. Coursera. "Algorithms Specialization." Diakses dari: https://www.coursera.org
7. Sumber: Medium https://share.google/EPk5B27EiSZFQClXw.